

React interview question bank

1. Core Fundamentals

Q1. What is React?

Answer:

React is a **JavaScript library** for building **user interfaces**, especially **component-based** UIs for single-page applications. It focuses only on the **view layer** and is maintained by Meta and a community of developers. It uses **declarative** programming, **components**, and a **virtual DOM** to efficiently update the UI.

Q2. What are the main features of React?

Answer:

- **JSX** – JavaScript syntax extension that looks like HTML.
 - **Components** – UI is broken into reusable components.
 - **Virtual DOM** – Efficiently updates the real DOM.
 - **Unidirectional data flow** – Data flows from parent to child.
 - **Hooks** – Functions that let you use state and lifecycle in function components.
 - **Declarative UI** – You describe *what* UI should look like, React takes care of *how*.
-

Q3. What is JSX and why is it used?

Answer:

JSX (JavaScript XML) is a **syntax extension** that allows you to write HTML-like code inside JavaScript. React uses JSX because:

- It is **more readable** and closer to UI structure.
- It allows **compile-time checks**.
- Under the hood, JSX is compiled to `React.createElement(...)` calls.

Example:

```
const element = <h1>Hello, world!</h1>;
```

Compiles roughly to:

```
const element = React.createElement('h1', null, 'Hello, world!');
```

Q4. Is JSX mandatory in React?

Answer:

No. You can write React components using `React.createElement`. JSX is just syntactic sugar. But in real-world apps, JSX is almost always used because of **developer productivity** and readability.

Q5. What is the Virtual DOM?

Answer:

The **Virtual DOM** is a **lightweight in-memory representation** of the real DOM. When state changes:

1. React renders a new virtual DOM tree.
2. It compares it with the previous virtual DOM (diffing).
3. It calculates the **minimal set of changes**.
4. It updates only the necessary parts in the **real DOM**.

This improves performance over manipulating the DOM directly for every change.

Q6. What is the difference between Virtual DOM and Real DOM?

Answer:

- **Real DOM:**
 - Direct representation of the UI in the browser.
 - Updating it frequently is slow.
 - **Virtual DOM:**
 - JavaScript object representation of the UI.
 - Cheap to create and diff.
 - React batches changes and applies minimal updates to the real DOM.
-

Q7. What is a component in React?

Answer:

A component is a **reusable piece of UI**. It can be:

- A **function** (function component)
- Or a **class** (class component, older style)

Each component:

- Accepts **props** (inputs).
 - May maintain **state** (internal data).
 - Returns UI (JSX) describing what should appear on screen.
-

Q8. Difference between Functional and Class components?

Answer:

Functional Components:

- Plain JS functions returning JSX.
- Use **Hooks** for state and lifecycle.
- Simpler, recommended for modern React.

Class Components:

- ES6 classes extending `React.Component`.
 - Use `this.state`, `this.setState()`, lifecycle methods (`componentDidMount`, etc.).
 - Not recommended for new code; mostly legacy.
-

Q9. What do you mean by “declarative” in React?

Answer:

Declarative means you **describe the UI for a given state**, and React figures out **how** to update the DOM efficiently. You don't imperatively manipulate DOM nodes (like `document.getElementById(...).innerHTML = ...`); you simply update the state and let React handle the DOM.

Q10. What is a Single Page Application (SPA)?

Answer:

An SPA is an application that:

- Loads a **single HTML page** initially.
- Updates the **view dynamically** using JavaScript (React, etc.) without full page reloads.
- Uses routing on the client side (e.g., React Router) to switch views.

Q11. What are props in React?

Answer:

Props (properties) are **inputs** to a component:

- Passed from **parent to child**.
 - **Read-only** (immutable inside the child).
 - Used to parameterize components.
-

Q12. What is state in React?

Answer:

State is **internal data** of a component:

- Managed by the component itself.
 - Can change over time.
 - When state changes, React **re-renders** the component.
-

Q13. Can you modify props inside a child component?

Answer:

No. Props are **read-only**. You can:

- Read them.
 - Derive state from props (carefully).
 - Or call a **callback function** passed via props to ask parent to update something.
-

Q14. What is the key prop in React and why is it important?

Answer:

key is a special prop used when rendering lists. It helps React **identify which items changed, added, or removed**.

Best practice:

- key should be **stable, unique, and predictable** for that list item (e.g., id).
- Avoid using array index as key unless list is static and never reorders.

Q15. What happens if you use index as key in a list?

Answer:

Using index as key can cause:

- Incorrect reuse of DOM nodes.
- Weird UI bugs (input values jumping between items).
- Poor performance during reordering.

React may mismatch items if the list order changes.

Q16. What is React.Fragment and when do you use it?

Answer:

`<React.Fragment>` or shorthand `<>...</>` allows you to **group multiple elements without adding extra DOM nodes**.

```
return (  
  <>  
    <h1>Title</h1>  
    <p>Description</p>  
  </>  
);
```

Useful when a component must return a single parent element, but you don't want extra `<div>` wrappers.

Q17. What is the role of ReactDOM.createRoot / ReactDOM.render?

Answer:

It **mounts the React component tree into the actual DOM**.

React 18+:

```
import { createRoot } from 'react-dom/client';
```

```
const root = createRoot(document.getElementById('root'));
```

```
root.render(<App />);
```

Before React 18:

```
ReactDOM.render(<App />, document.getElementById('root'));
```

Q18. What is the difference between controlled and uncontrolled components?

Answer:

Controlled Component:

- Form input is **controlled by React state**.
- Input's value is bound to state and updated via onChange.

Uncontrolled Component:

- Form input manages its own internal state.
 - You use ref to read its value when needed.
-

Q19. What is React.StrictMode?

Answer:

`<React.StrictMode>` is a helper component used in development:

- Highlights potential problems.
 - Double-invokes some lifecycle methods and useEffect setup functions to detect side-effect issues (development only).
 - Does not affect production behavior.
-

Q20. What is the difference between React and Angular / Vue at a high level?

Answer:

- **React:** Library for UI, focuses on view layer; unopinionated; uses JSX.
 - **Angular:** Full framework; strongly opinionated; TypeScript-first.
 - **Vue:** Progressive framework; templates + options API/Composition API; more batteries-included than React but less than Angular.
-

2. Components, Props, State (Q21–Q40)

Q21. How do you create a functional component?

```
function Greeting(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

Or with arrow function:

```
const Greeting = ({ name }) => <h1>Hello, {name}</h1>;
```

Q22. How do you pass props to a component?

```
<Greeting name="Hari" />
```

Inside Greeting, you receive them as props or destructured.

Q23. What is default props in React?

Answer:

Used to set default values when a prop is not provided.

```
function Button({ label }) {  
  return <button>{label}</button>;  
}
```

```
Button.defaultProps = {  
  label: "Click me",  
};
```

In modern React with TypeScript, more common to handle via default parameter:

```
function Button({ label = "Click me" }) {  
  return <button>{label}</button>;  
}
```

Q24. How do you conditionally render elements in React?

Common patterns:

```
{isLoggedIn ? <Dashboard /> : <Login />}
```

```
{isAdmin && <AdminPanel />}
```

Or with functions that return different JSX based on conditions.

Q25. How do you update state in a class component?

```
this.setState({ count: this.state.count + 1 });
```

When depending on previous state:

```
this.setState(prev => ({ count: prev.count + 1 }));
```

Q26. Why should you not modify state directly (e.g., state.count++)?

Answer:

- It **doesn't trigger re-render**.
- Breaks React's expectations.
- Can lead to inconsistent UI and bugs.

Always use `setState` (class) or state updater (hooks).

Q27. Can you explain lifting state up?

Answer:

When two or more components need to **share the same data**, you:

- Move state to their **closest common parent**.
- Pass it down as props.
- Also pass callbacks down to update it.

This is called **lifting state up**.

Q28. What is prop drilling?

Answer:

Prop drilling is passing props **through multiple levels** of components just so a deeply nested component can use them. This can make components less reusable and harder to maintain.

Solution: **Context API**, global stores (Redux, Zustand), or refactor.

Q29. What is composition in React?

Answer:

Composition means building complex UIs by **combining components** rather than inheritance.

Example: Passing components as children:

```
<Card>

  <Profile />

</Card>
```

Q30. When would you use children prop?**Answer:**

Use props.children when you want a component to accept **arbitrary nested JSX**.

```
function Card({ children }) {

  return <div className="card">{children}</div>;

}
```

Q31. How can you prevent a component from re-rendering unnecessarily?

High-level answer:

- Use React.memo for functional components.
- Use shouldComponentUpdate or PureComponent in class components.
- Use memoized values (useMemo) and callbacks (useCallback).
- Avoid creating new objects/arrays inline if not necessary.

(Deeper performance questions come later.)

Q32. What is displayName in React components?**Answer:**

displayName is a string used in **debugging messages** and React DevTools to identify components more clearly, especially for higher-order components.

```
MyComponent.displayName = "MySpecialComponent";
```

Q33. What are controlled components in detail?**Answer:**

- Form inputs whose **value** is driven by **React state**.

- React is the “single source of truth.”
- Every change triggers onChange which updates state, which re-renders the input.

Pros: Validation, formatting, enabling/disabling easily.

Cons: More re-renders if not optimized.

Q34. What is a pure component?

Answer:

React.PureComponent is a class component that implements **shallow comparison** of props and state in shouldComponentUpdate.

Re-renders only when shallow comparison shows differences.

Q35. Difference between React.Component and React.PureComponent?

Answer:

- React.Component re-renders by default whenever setState is called.
 - React.PureComponent does a shallow comparison of props and state and may skip re-render if unchanged.
-

Q36. Can functional components be “pure”?

Answer:

Yes, using React.memo, you can wrap a functional component so that it only re-renders when props change (shallow comparison).

```
const MyComponent = React.memo(function MyComponent(props) { ... });
```

Q37. What are HOCs (Higher-Order Components)?

Answer:

An HOC is a function that **takes a component and returns a new component**, adding extra behavior.

Example:

```
function withLogger(WrappedComponent) {  
  return function (props) {  
    console.log("Rendering with props", props);
```

```
    return <WrappedComponent {...props} />;  
  };  
}
```

Nowadays, many patterns that used HOCs are replaced by **Hooks**.

Q38. What are render props?

Answer:

A pattern where a component takes a function as a prop, and uses that function to know **what to render**.

```
<DataProvider  
  render={({data}) => <Chart data={data} />}  
/>
```

Q39. Why did React move towards hooks instead of classes, HOCs, render props?

Answer:

Hooks:

- Make it easier to **reuse logic** without HOCs and render prop nesting.
 - Simplify **state and side-effect handling**.
 - Avoid this confusion.
 - Make code more **composable and testable**.
-

Q40. What is a key difference in how class and functional components manage state?

Answer:

- **Class:** this.state is an object; this.setState merges partial updates.
 - **Function:** useState holds any type; calling setter **replaces** the state value.
-

3. Hooks (Q41–Q80)

Q41. What are Hooks in React?

Answer:

Hooks are special functions that let you “**hook into**” **React features** (state, lifecycle, context, etc.) from **function components**.

Examples: `useState`, `useEffect`, `useContext`, `useReducer`, `useMemo`, `useCallback`, `useRef`.

Q42. What are the rules of Hooks?

Answer:

1. Only call Hooks **at the top level** of your React function (not inside loops, conditions, nested functions).
2. Only call Hooks **from React functions** (function components or custom hooks), not from regular JS functions.

These rules ensure React can correctly track hook calls.

Q43. Explain `useState`.

Answer:

`useState` lets you **add state** to function components.

```
const [count, setCount] = useState(0);
```

- `count` is current state.
 - `setCount` updates state and triggers re-render.
 - Initial value passed to `useState` is used only on first render.
-

Q44. How does React batch state updates?

Answer:

Inside event handlers, React usually **batches multiple state updates** into a single re-render for performance. In React 18+, batching is more automatic even across async boundaries (e.g. promises, timeouts in some cases).

Q45. What if you update state based on previous state?

Answer:

Use the **functional updater form**:

```
setCount(prev => prev + 1);
```

This avoids stale state issues and is safer when multiple updates occur.

Q46. Explain `useEffect`.

Answer:

`useEffect` lets you perform **side effects** (data fetching, subscriptions, DOM manipulations) after render.

```
useEffect(() => {  
  // side effect  
  
  return () => {  
    // cleanup  
  };  
}, [dependencies]);
```

- Runs after render.
 - Cleanup runs before next effect or on unmount.
-

Q47. What are common mistakes with `useEffect`?

Answer:

- Missing dependencies, causing stale data.
 - Including too many dependencies, causing infinite loops.
 - Doing heavy computations directly in effect without memoization.
 - Not cleaning up subscriptions/events, causing memory leaks.
-

Q48. How do the `useEffect` dependencies work?

Answer:

- `[]` – runs only once after initial render.
- `[a, b]` – runs when a or b changes.
- No array – runs after every render.

React compares dependency array values **shallowly**.

Q49. What is useRef used for?

Answer:

useRef returns a **mutable object** whose `.current` property persists across renders and changing it does not cause re-render.

Used for:

- Storing DOM references.
- Storing mutable values like timers, previous values.

Q50. What is the difference between useRef and useState?

Answer:

- `useState` triggers re-render when state changes.
- `useRef` does not trigger re-render when `.current` changes. It's like a **box** to store mutable values.

Q51. Explain useMemo.

Answer:

`useMemo` **memoizes** a computed value to avoid expensive recalculations on every render.

```
const result = useMemo(() => expensiveFn(data), [data]);
```

Only recomputes if data changes.

Q52. When should you use useMemo?

Answer:

- When you have **expensive computations**.
- When you need referential equality to prevent unnecessary re-renders (e.g. passing memoized objects as props).

But don't overuse it; computation vs.memo overhead tradeoff.

Q53. Explain useCallback.

Answer:

useCallback memoizes **callback functions** so they maintain stable references across renders.

```
const handleClick = useCallback(() => {  
  // logic  
}, [deps]);
```

Used with React.memo or when passing callbacks to child components to avoid re-renders.

Q54. Difference between useMemo and useCallback?

Answer:

- useMemo memoizes **results** of a function.
- useCallback memoizes the **function itself**.

useCallback(fn, deps) is equivalent to useMemo(() => fn, deps).

Q55. Explain useReducer.

Answer:

useReducer is an alternative to useState for managing **complex state logic**.

```
const [state, dispatch] = useReducer(reducer, initialState);
```

reducer(state, action) returns new state.

It's useful when you have multiple related state transitions or want Redux-like patterns.

Q56. When should you prefer useReducer over useState?

Answer:

- When state is **complex or nested**.
 - When multiple state updates depend on one another.
 - When you want **predictable state transitions** via actions.
-

Q57. What is a custom Hook?

Answer:

A custom hook is a **reusable function** that uses other hooks inside. It encapsulates logic and can be reused across components.

Naming convention: must start with use (e.g., useAuth, useFetch).

Q58. Example of a simple custom hook.

```
function useWindowWidth() {  
  const [width, setWidth] = useState(window.innerWidth);  
  
  useEffect(() => {  
    const handleResize = () => setWidth(window.innerWidth);  
    window.addEventListener("resize", handleResize);  
    return () => window.removeEventListener("resize", handleResize);  
  }, []);  
  
  return width;  
}
```

Use:

```
const width = useWindowWidth();
```

Q59. What is useLayoutEffect and how is it different from useEffect?**Answer:**

- useEffect runs **after painting** the screen.
- useLayoutEffect runs **synchronously after DOM mutations but before the browser paints**.

useLayoutEffect is used when you need to **measure layout** or synchronously re-render before the user sees the UI (e.g., to avoid flicker). Overuse can hurt performance.

Q60. What is useImperativeHandle?

Answer:

Used with `forwardRef` to customize the value that is exposed to parent components when they use refs.

```
useImperativeHandle(ref, () => ({  
  focus: () => inputRef.current.focus(),  
}));
```

Q61. What is `useTransition`?**Answer:**

`useTransition` (React 18 concurrent features) lets you mark some state transitions as **non-urgent** (transitions) so React can **keep the UI responsive**.

Q62. What is `useDeferredValue`?**Answer:**

`useDeferredValue` lets you **defer** a value, allowing React to show previous value while new one is being calculated. Useful for search, filtering, where you want UI to stay responsive.

Q63. What is `useId`?**Answer:**

`useId` generates a **unique, stable ID** that works in both client and server rendering for associating labels and inputs, etc.

```
const id = useId();
```

Q64. What happens if you break the rules of hooks?**Answer:**

React's internal mechanism for tracking hook calls gets confused. You may see errors like:

Invalid hook call. Hooks can only be called inside of the body of a function component.

Or weird, unpredictable behavior.

Q65. Why must hooks be called in the same order on every render?

Answer:

React uses the **call order** to associate hook state with components. If order changes (e.g., calling a hook inside condition), React assigns wrong state to the wrong hook.

Q66. Can you use hooks inside loops or conditions?**Answer:**

No. Hooks must always be called **at the top level** of the component.

Q67. How do you handle async/await in useEffect?**Answer:**

You cannot make the effect callback itself async (it returns a cleanup function, not a Promise). Instead:

```
useEffect(() => {  
  let isMounted = true;  
  
  async function fetchData() {  
    const data = await getData();  
    if (isMounted) setData(data);  
  }  
  
  fetchData();  
  
  return () => { isMounted = false; };  
}, []);
```

Q68. What is an effect cleanup function?**Answer:**

The function returned from useEffect:

```
useEffect(() => {  
  const id = setInterval(...);  
  
  return () => clearInterval(id);  
}, []);
```

Used to clean up subscriptions, timers, event listeners.

Q69. Explain dependency array pitfalls in useEffect.

Answer:

- Leaving dependencies out can cause **stale values** or inconsistent behavior.
 - Including unstable references (like new functions/objects created every render) can cause frequent re-runs.
 - To fix, memoize functions/objects via useCallback / useMemo.
-

Q70. How do you avoid infinite loops in useEffect?

Answer:

- Ensure setState inside effect doesn't cause dependency to change in a way that keeps re-triggering.
- Use correct dependency arrays.
- Memoize callbacks/values if they're used as dependencies.

Short conceptual ones

Q: Can hooks be used in class components?

A: No.

Q: What is useContext used for?

A: To read values from a React context without using <Context.Consumer>.

Q: Can you call a custom hook from another custom hook?

A: Yes, as long as rules of hooks are followed.

Q: Should useState initial value be computed inside or outside?

A: For expensive initialization, use lazy initial state: useState(() => expensiveInit()).

Q: What is lazy initialization in useState?

A: Passing a function to useState which runs only on first render.

Q: When does useEffect run relative to paint?

A: After paint.

Q: Can useEffect run before first paint?

A: No; that's useLayoutEffect.

Q: Do hooks work with server components?

A: Some do, some don't (depends on server/client component boundaries in modern React).

Q: What is "state closure" in hooks?

A: When effect or callback captures old variables because dependencies aren't declared properly.

Q: How to store previous props or state values?

A: Use useRef combined with useEffect.

Q81. What are lifecycle methods in React (class components)?

React class components have three lifecycle phases:

Phase	Methods
Mounting	constructor, getDerivedStateFromProps, render, componentDidMount
Updating	shouldComponentUpdate, render, getSnapshotBeforeUpdate, componentDidUpdate
Unmounting	componentWillUnmount

Hooks replace these with useEffect, useLayoutEffect, useRef, etc.

Q82. Equivalent of componentDidMount in Hooks?

```
useEffect(() => {  
  // runs once after initial render  
}, []);
```

Q83. Equivalent of componentWillUnmount in Hooks?

Use cleanup function:

```
useEffect(() => {  
  return () => {  
    console.log("Component unmounted");  
  };  
}, []);
```

Q84. Equivalent of componentDidUpdate in Hooks?

```
useEffect(() => {
```

```
console.log("Updated when count changes");  
}, [count]));
```

Q85. Does `useEffect` run before or after paint?

After paint — meaning UI appears first, then effect executes.

Q86. What is `useLayoutEffect` used for?

Runs **before browser paints**. Used when UI measurement or synchronous layout adjustments are required.

If you modify DOM based on measurement, use **`useLayoutEffect`** to avoid flicker.

Q87. Why does React batch updates?

To improve performance — React waits and groups multiple state updates into a single re-render.

Q88. How many times can React re-render in Strict Mode?

React intentionally **double-invokes rendering in development mode** to identify side-effect bugs.

It does NOT happen in production.

Q89. What causes unnecessary re-renders?

- Passing new object/array literals to children.
 - Inline anonymous functions.
 - Incorrect dependencies in `useEffect`.
 - Changing props values unnecessarily.
-

Q90. How do you stop unnecessary renders in functional components?

- Wrap component with `React.memo`.
- Use `useCallback` to stabilize function props.
- Use `useMemo` for computed values passed as props.

Q91. What is reconciliation in React?

Reconciliation is the process React uses to determine what changed in the virtual DOM. It uses:

- **Diffing algorithm**
 - **Keys** for list identification
-

Q92. What is the “Diffing Algorithm”?

React compares old vs new virtual DOM:

- If type changes → re-render whole subtree
 - If same type → compare props and update only changed nodes
 - For children (lists), React uses **keys** for efficient matching
-

Q93. What happens if a key changes between renders?

React treats the element as a **new component**.

Example: input typing may reset because the DOM node gets recreated.

Q94. Why shouldn't you mutate state directly?

Because React won't detect the mutation, no re-render will occur, leading to inconsistencies.

Wrong:

```
state.count++
```

Correct:

```
setCount(prev => prev + 1);
```

Q95. What is the difference between reconciliation and rendering?

Rendering	Reconciliation
Building the virtual DOM snapshot	Comparing snapshots to determine required DOM changes

Q96. What triggers a re-render?

- State update (useState, useReducer)
 - Parent re-render passing new props
 - Context update
 - Forced render (forceUpdate in classes)
-

Q97. Does updating state with the same value trigger re-render?

No. React internally compares current and next value and skips update if unchanged.

Q98. Does React update the DOM on every re-render?

No. It only updates changed parts after diffing.

Q99. What are side effects in React?

Operations occurring **outside UI rendering**, such as:

- Fetching API data
 - Adding event listeners
 - Logging
 - DOM manipulation
-

Q100. Why shouldn't we put heavy computations inside render or component body?

Because React **calls render often** — heavy operations harm performance. Use **useMemo** for optimization.

5. Performance Optimization & Patterns (Q101–Q135)

Q101. What is React.memo()?

A higher-order function that memoizes component output, preventing re-render if props didn't change.

```
export default React.memo(MyComponent);
```

Q102. When NOT to use React.memo()?

Avoid when:

- Component is cheap to render
 - Props change frequently
 - Overhead of shallow comparison > savings
-

Q103. How do useMemo and React.memo work together?

- React.memo prevents re-render.
- useMemo prevents re-calculating expensive values.

Often used together.

Q104. What does useCallback solve?

Prevents new function reference on every render, useful when passing callback to memoized children.

Q105. What is lazy loading in React?

React loads components **only when needed**, improving initial load time.

```
const Dashboard = React.lazy(() => import("./Dashboard"));
```

Q106. What is Suspense?

Used to show fallback UI while a lazy component loads:

```
<Suspense fallback={<Loader />}>
```

```
  <Dashboard />
```

```
</Suspense>
```

Q107. What are render props vs children props?

Both are patterns for passing functions as children, but:

Render props

component expects a prop named render

Children as function

function passed as children

Q108. What is tree shaking?

Removing unused code during bundling. If you don't import something, it isn't included.

React builds benefit from bundlers like **Webpack, Vite, SWC**.

Q109. What is hydration in React?

When React attaches event listeners to **server-rendered HTML** during SSR.

Q110. What is React Server Component (RSC)?

Introduced with React 18 and Next.js — allows running component code on the server, sending rendered UI to the client, reducing bundle size.

Q111. How do keys affect reconciliation?

Correct keys help React re-match components between renders.
Incorrect keys → wasted DOM operations and UI bugs.

Q112. How do you implement infinite scrolling?

Usually by:

- Using Intersection Observer API
 - Triggering API calls when sentinel enters viewport
 - Managing pagination state
-

Q113. What is windowing/virtualization in React?

Rendering only visible list items instead of entire huge list (like 10,000 items).

Libraries: react-window, react-virtualized.

Q114. What is batching in React 18?

React batches state updates even in promises, timeouts, async events — improving performance.

Q115. What is shallow comparison?

Comparison checking only first-level properties (not nested deeply).

Used by:

- `React.memo`
 - `PureComponent`
-

Q116. Why shallow compare is important?

Deep comparison is expensive — shallow compare balances correctness and performance.

Q117. What are controlled re-renders?

Limiting re-rendering using:

- `useCallback`
 - `useMemo`
 - `React.memo`
 - Context splitting
 - Local state instead of lifting unnecessarily
-

Q118. What is stale state?

When closures use outdated state due to missing dependencies.

Q119. How to avoid stale closures?

Correctly define dependency arrays; use functional updates.

Q120. What is a synthetic event in React?

React wraps browser events in a cross-browser compatible wrapper.
React reuses event objects for performance.

Q121. What happens if you console.log event object after asynchronous use?

It prints **nullified values**, because React pools event objects.

To fix:

```
event.persist();
```

Q122. What is debouncing and throttling?

Optimizing repeated actions (scroll, input):

- **Debounce:** wait until user stops typing
 - **Throttle:** execute at fixed intervals
-

Q123. How to debounce input in React?

Use useEffect + timer or external libraries like lodash.

Q124. What is SSR (Server-Side Rendering)?

Rendering React on the server, sending HTML to client to improve SEO and FCP.

Q125. What is CSR (Client Side Rendering)?

Browser downloads JS and renders the UI locally. Initial load slower, interactions faster.

Q126. Which React framework supports SSR?

Next.js (most common), Remix also supports advanced SSR.

Q127. What is hydration mismatch?

When server-rendered HTML differs from client-side rendering → warnings or errors.

Q128. How to reduce bundle size?

- Lazy load components
 - Use code splitting
 - Enable tree shaking
 - Use React Server Components (Next.js)
-

Q129. What is bundle splitting?

Splitting app code into smaller chunks loaded on demand.

Q130. What is Suspense boundary behavior?

Each Suspense wrapper handles only its lazy components — rest of UI remains interactive.

Q131. Why is React efficient for UI diffing?

It assumes:

- Different component types → replace
- Same type → diff props and children
- Lists → compare using keys

This heuristic avoids expensive deep DOM comparisons.

Q132. Why is mutation bad in React?

Mutations prevent change detection → React won't re-render.

Q133. Can React skip rendering if state updates with same reference?

Yes — React compares old and new values. If identical, no render occurs.

Q134. Why avoid passing inline functions to memoized children?

Because inline functions create **new function identity** each render → memo comparison fails → triggers re-render.

Use useCallback.

Q135. What are Suspense boundaries used for besides lazy loading?

In future features — **data fetching**, streaming, server components, concurrent UI.

Q136. What is the Context API in React?

The Context API allows you to share data across multiple components **without prop drilling**. It provides:

- createContext()
- Provider to supply value
- useContext() to consume it

Q137. When should you use context?

Use context when:

- Many deeply nested components need access.
- The data is global in nature (ex: authentication, theme, language).

Avoid context for rapidly changing UI values — may trigger unnecessary re-renders.

Q138. How do you create and use context?

```
const ThemeContext = React.createContext();
```

```
function App() {  
  return (  
    <ThemeContext.Provider value="dark">  
      <Navbar />  
    </ThemeContext.Provider>  
  );  
}
```

```
function Navbar() {
```

```
const theme = useContext(ThemeContext);  
return <div>Theme: {theme}</div>;  
}
```

Q139. What is the difference between `Context.Provider` and `useContext()`?

- `Provider` supplies the data.
 - `useContext()` allows components to consume that data.
-

Q140. Do all components inside a context re-render when the value changes?

Yes — by default **all consumers** re-render when context value updates.

To avoid unwanted re-renders, you can:

- Split contexts
 - Memoize context values
 - Use selectors (e.g., Zustand or Redux)
-

Q141. Should you store large or frequently changing state in `Context`?

No. `Context` is not built for high-frequency updates — use state managers like:

- Zustand
 - Redux Toolkit
 - Jotai
 - Recoil
-

Q142. What is `Redux`?

`Redux` is a predictable state container based on:

- Single global store
 - Immutability
 - Pure reducers
 - Unidirectional flow
-

Q143. What is Redux Toolkit?

Redux Toolkit (RTK) is the **recommended modern way** to use Redux. It simplifies:

- Boilerplate
 - Actions
 - Reducers
 - Async requests (createAsyncThunk)
-

Q144. What are the core principles of Redux?

1. **Single source of truth**
 2. **State is read-only**
 3. **Changes made through pure reducers**
-

Q145. What is an action in Redux?

A plain JavaScript object describing what happened:

```
{ type: "INCREMENT" }
```

Q146. What is a reducer?

A pure function that takes current state + action and returns new state:

```
function counter(state, action) {  
  switch(action.type) {  
    case 'INC': return state + 1;  
    default: return state;  
  }  
}
```

Q147. What is a store?

The central container holding application state in Redux.

Q148. What is useSelector and useDispatch?

- useSelector() → read state from store

- useDispatch() → dispatch actions to update state
-

Q149. What is middleware in Redux?

Middleware intercepts actions before they reach the reducer. Common for:

- Logging
- API requests
- Authentication

Example: Redux Thunk, Redux Saga.

Q150. What is Redux Thunk?

Middleware allowing action creators to return **functions** instead of objects, enabling async logic:

```
const fetchUsers = () => async (dispatch) => {  
  const res = await fetch("/users");  
  dispatch(setUsers(res.json()));  
};
```

Q151. What is Zustand?

A lightweight state management library replacing Redux for simplicity, without boilerplate.

Q152. What is Recoil?

A React state management tool with **atoms** (state units) and **selectors** (derived state).

Q153. What is the difference between Redux and Context API?

Feature	Redux	Context API
Purpose	Global complex state	Avoid prop drilling
Performance	Optimized	Re-renders consumers

Feature	Redux	Context API
Tools	Actions, reducers	Provider + useContext
Async handling	Built-in patterns	Manual

Q154. Should Redux replace React state?

No — use Redux only when:

- Many components need shared data
 - State logic is complex
 - Debugging & predictability matter
-

Q155. Can you use Context + useReducer as a Redux alternative?

Yes — for medium complexity apps.

Q156. How to avoid re-renders with Context?

- Split contexts
- Memoize provider values:

```
const value = useMemo(() => ({ theme, setTheme }), [theme]);
```

Q157. Is context asynchronous?

Yes — like state updates, context update triggers rerender in next reconciliation cycle.

Q158. Can context be used with server-side rendering?

Yes. Frameworks like Next.js commonly use context for auth, themes, preferences.

Q159. When NOT to use context?

- When data changes rapidly (scroll position, input typing)
 - When only one or a few components need the data
-

Q160. What is Selector Pattern in state management?

Selecting only a small part of global state instead of full context value to optimize rendering.

Q161. What is React Router?

React Router is a routing library enabling:

- Multi-page feel in SPAs
 - URL management
 - Navigation without full reloads
-

Q162. What are the main components of React Router v6?

- `<BrowserRouter>`
 - `<Routes>`
 - `<Route>`
 - `useNavigate`
 - `useParams`
 - `useLocation`
-

Q163. How do you define a basic route?

```
<Routes>

  <Route path="/home" element={<Home />} />

</Routes>
```

Q165. How do you read URL parameters?

```
const { id } = useParams();
```

Q166. How do you pass state during navigation?

```
navigate("/profile", { state: { userId: 10 } });
```

Retrieve:

```
const location = useLocation();
```

```
console.log(location.state.userId);
```

Q167. What is nested routing?

Routing pages inside parent layouts:

```
<Route path="/dashboard" element={<Dashboard />}>  
  <Route path="reports" element={<Reports />} />  
</Route>
```

Q168. How to define a wildcard (404) route?

```
<Route path="*" element={<NotFound />} />
```

Q169. What is the difference between <Link> and <NavLink>?

- <Link>: navigation only
 - <NavLink>: adds active class based on URL match
-

Q170. How do you implement protected routes?

Using conditions inside routes:

```
<Route element={user ? <Dashboard /> : <Login />} />
```

Q171. What is Outlet used for?

Placeholder for nested route components.

Q172. How do you handle search params?

```
const [params] = useSearchParams();  
params.get("query");
```

Q173. Does React Router replace server routing?

No — it works **client-side**, server must still respond with the SPA HTML for all routes.

Q174. Can React Router preload data?

Yes — newer versions and frameworks like Remix/Next.js support route-level loaders.

Q175. What is difference between hash routing vs browser routing?

BrowserRouter	HashRouter
clean URLs (/home)	hash URLs (#/home)
requires server support	works anywhere
better SEO	not SEO-friendly

Q176. How do you redirect in React Router?

`navigate("/login");`

Or `<Navigate to="/login" />`.

Q177. How do you block navigation?

Using Prompt (older versions) or custom logic with guarded routes.

Q178. Can React Router lazy load routes?

Yes:

```
<Route path="/admin" element={
  <Suspense fallback=<Loader />>
    <Admin />
  </Suspense>
}/>
```

Q179. How do you animate route transitions?

Using libraries such as:

- Framer Motion
- React Transition Group

Q180. Does React Router support SSR frameworks?

Yes — frameworks like **Next.js**, **Remix**, **Gatsby** handle routing with SSR capabilities.

Q181. What are controlled form inputs?

Input values controlled via React state:

```
<input value={name} onChange={(e) => setName(e.target.value)} />
```

Q182. What are uncontrolled components?

Input manages its own state; React reads via ref.

Q183. When to prefer uncontrolled components?

- Simple forms
 - File uploads
 - Non-interactive values
-

Q184. How to manage multiple input fields?

Using dynamic state keys:

```
setForm({ ...form, [e.target.name]: e.target.value });
```

Q185. How do you validate forms in React?

Options:

- Manual validation
 - Libraries (Formik, React Hook Form, Yup, Zod)
-

Q186. What is Formik?

A form library simplifying:

- Field handling
 - Errors
 - Touch tracking
 - Validation schemas
-

Q187. What is React Hook Form?

A performant form manager using uncontrolled inputs and refs, minimizing re-renders.

Q188. How do you show input validation errors?

Track validation state and display conditionally:

```
{error && <span>{error}</span>}
```

Q189. How do you handle async validation (e.g., username check)?

Inside submit handler or effect, with debouncing.

Q190. How to submit a form programmatically?

Call handleSubmit() manually or ref.submit().

Q191. How to handle dynamic form fields (add/remove)?

Store field list in array and map to UI.

Q192. How to handle file inputs?

Use uncontrolled input with ref and FormData().

Q193. How do you disable submit until form is valid?

Use state conditions:

```
<button disabled={!isValid}>Submit</button>
```

Q194. What is optimistic UI update?

Updating UI before server confirmation — improves UX.

Q195. How do you reset a form?

Using:

- setState(initialState)

- `form.reset()`
 - Form library reset API
-

Q196. What is form serialization?

Converting form data into JSON or FormData.

Q197. How to auto-save form input?

Use `useEffect` with debouncing to save to:

- `LocalStorage`
 - Backend autosave endpoint
-

Q198. How to store form data between reloads?

Use `LocalStorage` with hydration on mount.

Q199. How do you prevent accidental form submission?

`event.preventDefault()` in `onSubmit`.

Q200. How to build multi-step forms in React?

Use:

- State machine
- Step component logic
- Global form data store (context/Redux)

Q201. What are Server Components (RSC) in React 18 and why do they matter?

Server Components allow React to run logic on the **server**, not the browser. They:

- Reduce bundle size (no JS sent for that component)
- Improve performance
- Allow direct DB/API access without exposing credentials

Used heavily in **Next.js App Router**.

Q202. Can Server Components use hooks?

Hook	Allowed?
useState/useEffect	✗ No
useRef (non-DOM)	⚠ Limited
useContext (Server Context)	✓ Allowed
Custom async hooks	✓ Allowed

Because server components don't run in the browser, side-effect hooks don't apply.

Q203. What is hydration?

Hydration is when React attaches event listeners to HTML generated on the server. Used in SSR frameworks like **Next.js**, **Remix**, **Gatsby**.

Q204. What is partial hydration?

Hydrating only parts of UI that are interactive instead of entire page → improves performance.

Q205. What is progressive hydration / streaming?

React streams HTML in chunks and hydrates components **as they load**, improving first paint.

Q206. CSR vs SSR vs SSG vs ISR — differences?

Model	Rendering Location	Use Case
CSR	Browser	Dashboards, internal tools
SSR	Server (request-time)	Auth-based pages
SSG	Build-time	Blogs/docs
ISR	Build + scheduled regeneration	Content sites with updates

Q207. What is useSyncExternalStore?

React hook for integrating external state stores (Zustand/Redux) with React's concurrent rendering.

Q208. What is resource preloading / prefetching in React apps?

- **Preload:** load resources needed immediately
- **Prefetch:** load resources likely needed soon (e.g., next route)

Example in React Router:

```
<Link to="/admin" preload="intent">Admin</Link>
```

Q209. How do Suspense + data fetching work together?

React can suspend rendering until data resolves:

```
const data = await fetchData();  
return <Suspense fallback="loading...">{data}</Suspense>
```

Q210. What are transitions in React 18?

Used for marking non-urgent state updates:

```
const [isPending, startTransition] = useTransition();  
startTransition(() => setQuery(input));
```

This keeps UI responsive.

Q211. What is the main difference between controlled vs uncontrolled components?

Controlled = value stored in React state

Uncontrolled = value stored in DOM

Q212. What are portals in React?

Render a component **outside the normal parent DOM tree**, useful for modals, tooltips:

```
ReactDOM.createPortal(<Modal />, document.getElementById("root-modal"));
```

Q213. What is an error boundary?

A class component that catches JavaScript errors in child components:

`componentDidCatch(error, info)`

Functional equivalent uses libraries, upcoming Hook supports will change this.

Q214. Why don't hooks have error boundaries built-in?

Because hooks run inside render logic — React isolates error handling separately.

Q215. What is reconciliation failure?

When React incorrectly matches elements due to bad keys → UI shifts unexpectedly.

Q216. How does `React.lazy()` differ from dynamic ES imports?

`React.lazy()` integrates with `Suspense`, enabling fallback UI while loading.

Q217. What is anti-pattern in React? Give examples.

Examples:

- Using index as key
 - Storing derived values directly in state
 - Mutating props/state
 - Overusing context
-

Q218. Why is spreading props (`{...props}`) sometimes harmful?

It can unintentionally forward unnecessary data → harder debugging and performance issues.

Q219. Why is `useEffect` NOT for data fetching (modern guidance)?

React encourages async data fetching “outside render” using:

- Server Components
 - Suspense-based frameworks (Next.js loaders)
-

Q220. What are layout shifts and how to avoid them?

Sudden visual jumps due to delayed content. Avoid by:

- Setting fixed height for images
 - Using skeleton loaders
 - Correct DOM structure
-

Q221. Best practices for large-scale React apps?

- Modular folder structure
 - State normalization
 - Feature-based components
 - Avoid monolithic context
 - Lazy loading & route splitting
-

Q222. What is the container-presentational component pattern?

- **Presentational components** → UI only
- **Containers** → business logic, data fetching

Now mostly replaced by hooks.

Q223. What is an accessibility-friendly React UI?

- Use semantic HTML instead of <div>
 - Add aria-* attributes when needed
 - Focus management in modals
 - Keyboard navigation support
-

Q224. Explain stale closure with example.

```
useEffect(() => {  
  setTimeout(() => console.log(count), 2000);  
}, []);
```

Logs original value because dependencies missing.

Q225. What is useEvent (experimental)?

Stable callback reference that always sees latest state without re-render.

Q226. What is the difference between unit testing and integration testing in React?

Unit Test	Integration Test
Tests isolated logic/component	Tests interactions between components
Fast, granular	Slower, more realistic

Q227. What testing libraries are commonly used?

- **Jest** (test runner)
 - **React Testing Library**
 - **Cypress / Playwright** (E2E testing)
 - **Vitest** (Vite-native Jest alternative)
-

Q228. How do you test a button click?

```
fireEvent.click(screen.getByText("Submit"));  
expect(mockFn).toHaveBeenCalled();
```

Q229. What is screen in RTL?

A global object to query rendered DOM elements using accessibility-first APIs.

Q230. Why use Testing Library instead of Enzyme?

RTL focuses on **testing user behavior**, not component internals → aligned with modern React.

Q231. How do you mock API calls in tests?

Using `jest.mock()` or `msw` (Mock Service Worker).

Q232. What is mocking with MSW?

MSW intercepts real browser fetch/XHR and responds with mocked data. Best for integration tests.

Q233. How to test async UI updates?

```
await waitFor(() => expect(screen.getByText("Loaded")).toBeInTheDocument());
```

Q234. How do you test custom hooks?

Using `renderHook()` from RTL or React Testing Hooks library.

Q235. What is snapshot testing?

Captures rendered UI and compares against stored snapshot to detect unexpected UI changes.

Q236. Why is snapshot testing controversial?

Because many snapshots become large and developers often approve changes without reviewing.

Q237. How do you test routing behavior?

Using RTL's `MemoryRouter`:

```
render(<MemoryRouter initialEntries={['/login']}><App /></MemoryRouter>);
```

Q238. Should you test implementation details in React?

No — test behavior, not internal hooks or state.

Q239. A component re-renders too often — how do you debug it?

- Use React DevTools → Profiler
 - Check props changes
 - Wrap with `React.memo`
 - Fix dependency arrays
 - Avoid unnecessary object recreation
-

Q240. API calls firing multiple times — causes?

- Missing dependency array in useEffect
 - React Strict Mode double invoke
 - Refreshing state causing re-render
-

Q241. A modal closes unexpectedly on typing — why?

State lifting incorrectly or re-rendering parent resets modal state.

Q242. List rendering is slow — fix?

- Virtualization (react-window)
 - Stable keys
 - Memoized rows
-

Q243. Login page flashes wrong UI before redirect — solution?

- Skeleton screen
 - Auth state gating
 - Early routing guard
-

Q244. Form performance issue — why?

Controlled inputs cause re-renders → solution: Use React Hook Form or uncontrolled approach.

Q245. How do you handle large nested JSON UI?

- Normalize data
 - Componentize sections
 - Use pagination, lazy load
-

Q246. User sees stale UI after updating — why?

- Cached network response

- React memo blocking update
 - Derived state not updating
-

Q247. Why might WebSockets not update UI?

State update references reused or incorrect closures stopping re-render.

Q248. You need undo/redo — how implement?

Use **state history stack** with useReducer.

Q249. How do you implement dark mode persistently?

- Store preference in localStorage
 - Hydrate state on mount
 - Sync with system theme using matchMedia
-

Q250. Final: “How would you architect and structure a large-scale React app?”

Answer summary:

- Feature-based folder structure
- Lazy-loaded routes
- Context only for global stable state
- Zustand/Redux for shared dynamic state
- Custom hooks for business logic
- Strong typing (TS)
- SSR for SEO pages
- Code splitting, Suspense
- Testing coverage for critical flows